DocuVault

Secure Document Sharing & E-Signature Simulator

Project Documentation & 2-Week Development Plan

MERN Stack | React + Tailwind CSS | Node.js + Express | MongoDB

Skill-Testing Project | Difficulty: 7 / 10 | Target Duration: 14 Days

Prepared for: Pre-Capstone Skill Assessment

Scope note: DocuVault is naturally a 3–4 week project. To fit it into 14 focused days, this plan trims scope to one core advanced feature (secure token-based sharing + canvas signature capture + audit trail) and pushes version history to an optional stretch goal. Read the 'Scope & Realism Check' section before you start.

1. Project Overview

Objective

Build a full-stack web application where users can upload documents (contracts, agreements, forms), share them securely with specific people via a unique link, and collect a simulated e-signature from each recipient — with a complete audit trail showing who viewed and signed, and when.

Real-World Problem

Freelancers and small teams often need someone to review and 'sign off' on a document — a contract, a proposal, a design approval — but tools like DocuSign are overkill and expensive for occasional use. Email attachments get lost, there's no record of who actually opened or approved something, and there's no single place to track document status.

Target Users

- **Owner** — uploads a document, invites recipients, tracks status, views audit trail
- **Recipient / Signer** — opens a document via a secure link, reviews it, and signs it

Tech Stack

Layer	Technology
Frontend	React (JavaScript), Tailwind CSS, React Router, Canvas API
Backend	Node.js, Express.js, JWT Authentication, Multer
Database	MongoDB with Mongoose ODM
Storage	Local disk for dev (or Cloudinary/S3 as a stretch upgrade)
Tooling	Postman/Thunder Client, Git/GitHub, dotenv

Core Features

- User authentication (JWT) — any user can be a document Owner
- Upload documents (PDF/image) with validation on type and size
- Share a document with specific people via a unique, token-based link
- Recipient can view the document and sign it (typed name or drawn signature)
- Owner dashboard: document list with status (Pending / Partially Signed / Completed)
- Audit trail: log of every view and signature event, with timestamp

Advanced / Challenging Features (this plan builds these three)

- **Secure token-based access** — recipients access a document via a unique link without necessarily needing a full account, but only that link works for that document
- **Canvas-based signature capture** — draw a signature on an HTML canvas and save it as an image, placed at a specific position on the document

- **Audit trail system** — every view/sign action is logged with who, what, and when, and displayed as a timeline to the document owner

Stretch Goal (only if you finish early)

- Version history — if the owner re-uploads a corrected document, keep previous versions accessible and show what changed

2. System Architecture

Folder Structure

```
docuvault/
■■■ client/
■  ■■■ src/
■  ■  ■■■ components/ (documents, signing, audit, common)
■  ■  ■■■ context/ (AuthContext.jsx)
■  ■  ■■■ hooks/ (useAuth, useDocuments, useSignaturePad, useUploadProgress)
■  ■  ■■■ pages/
■  ■  ■■■ routes/
■  ■  ■■■ utils/
■  ■  ■■■ App.jsx
■  ■■■ tailwind.config.js
■■■ server/
■  ■■■ controllers/
■  ■■■ models/
■  ■■■ routes/
■  ■■■ middleware/ (auth.js, tokenAccess.js, errorHandler.js)
■  ■■■ uploads/ (stored documents, dev only)
■  ■■■ utils/ (generateAccessToken.js)
■  ■■■ server.js
■■■ README.md
```

Database Collections

Collection	Key Fields
users	name, email, passwordHash
documents	ownerRef, title, fileUrl, fileType, status (pending/partially_signed/completed), createdAt
recipients	documentRef, name, email, accessToken, status (invited/viewed/signed), signatureImageUrl, signedAt
auditlogs	documentRef, recipientRef, action (viewed/signed), timestamp, ipAddress (optional)

*Design note: **recipients** is kept as its own collection (not embedded in documents) because each recipient needs a unique access token and independent status — this is the relationship modeling decision worth explaining in an interview.*

Key API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login, returns JWT
POST	/api/documents	Upload a new document (Multer)
GET	/api/documents	List owner's documents (paginated, filterable by status)
GET	/api/documents/:id	Document details (owner view)

POST	/api/documents/:id/recipients	Invite a recipient (generates access token)
GET	/api/access/:token	Recipient opens document via secure link (logs 'viewed')
POST	/api/access/:token/sign	Recipient submits signature (logs 'signed')
GET	/api/documents/:id/audit	Full audit trail for a document (owner only)

3. Two-Week Development Plan

This plan assumes roughly 4 focused hours/day. To realistically fit a 7/10-difficulty project into 14 days, scope is deliberately trimmed: build ONE advanced feature set well (token-based sharing + canvas signatures + audit trail) rather than everything listed as possible. Week 1 builds a fully working, tested backend. Week 2 builds the frontend, integrates it, and polishes for demo.

Week 1 — Backend Foundation, Secure Sharing & Signing Logic

Day	Focus Area	Tasks	Deliverable
1	**Setup & Schema Design**	Init client/server folders. Set up Express server, MongoDB (Atlas) connection, env config. Install core deps (express, mongoose, dotenv, cors, bcrypt, jsonwebtoken, multer, crypto for tokens). Design schema: User, Document, Recipient, AuditLog.	Server running & connected to MongoDB. Schema diagram finalized.
2	**Authentication Backend**	Build User model. Implement register/login endpoints, bcrypt password hashing, JWT issuing. Write verifyToken middleware for owner-only routes.	Auth endpoints tested and working in Postman.
3	**Document Upload**	Build Document model. Implement upload endpoint with Multer (PDF/image validation, size limits). Store file, save metadata. List/get endpoints for owner's documents.	Owner can upload and list their own documents.
4	**Secure Sharing (Token Links)**	Build Recipient model. Implement invite-recipient endpoint that generates a unique, unguessable access token (crypto.randomBytes). Build GET /access/:token to fetch a document by token without requiring full login.	A recipient can be invited and access the document via a unique link.
5	**Audit Logging — Views**	Build AuditLog model. Every time /access/:token is hit, log a 'viewed' event with recipient, document, and timestamp. Update recipient status to 'viewed'.	Viewing a document via link is correctly logged.
6	**Signature Endpoint**	Build POST /access/:token/sign — accepts a signature image (base64 from canvas) or typed name, saves it, updates recipient status to 'signed', logs a 'signed' audit event, and recomputes the parent document's overall status.	Signing flow works end-to-end via API calls (tested in Postman).
7	**Backend Testing & Buffer**	Test every endpoint with edge cases: invalid token, expired/reused token, unauthorized access to another owner's document. Add centralized error-handling middleware. Clean commits, push to GitHub.	Backend considered DONE — stable, tested, version-controlled.

Week 2 — Frontend, Signature Capture, Audit UI & Deployment

Day	Focus Area	Tasks	Deliverable
8	**Frontend Setup & Auth UI**	Set up React + Tailwind + React Router. Build folder structure, AuthContext, useAuth hook. Build Login/Register pages and Owner Dashboard shell.	Working login/register flow with protected owner routes.
9	**Upload UI & Document Dashboard**	Build document upload form with progress indicator (useUploadProgress hook). Build dashboard listing owner's documents with status badges (Pending/Partially Signed/Completed).	Owner can upload documents and see them listed with live status.
10	**Invite Recipients UI**	Build 'Share' modal — add recipient name/email, call invite endpoint, display the generated shareable link. Show per-document recipient list with status.	Owner can invite recipients and copy their unique access links.
11	**Recipient Access Page (Public Link View)**	Build the page that opens via /access/:token — shows document content, recipient info, and a 'Sign' call-to-action. No login required for this page.	Recipient can open their link and view the document.
12	**Canvas Signature Capture**	Build a signature pad component using the Canvas API (draw with mouse/touch, clear, typed-name fallback). Convert canvas to image data, submit to the sign endpoint.	Recipient can draw or type a signature and submit it successfully.
13	**Audit Trail UI & Status Sync**	Build the audit trail timeline view (owner-only) showing every view/sign event with timestamps. Use useEffect to refresh document status after actions.	Owner can see a full, readable history of who viewed/signed and when.
14	**Polish, Optimization & Deployment**	Add loading/empty/error states everywhere. Memoize document list rendering. Responsive Tailwind pass. Write README. Deploy backend (Render/Railway) and frontend (Vercel/Netlify). Final end-to-end test with a real multi-recipient document.	Deployed, polished, demo-ready project with documentation.

If you fall behind: *cut the audit-trail UI polish before you cut the signature flow — the canvas signing experience is the feature most worth demoing in an interview. Version history (the stretch goal) should be the first thing dropped, not last.*

4. Scope & Realism Check

Be honest with yourself about this timeline. DocuVault touches file handling, security (token-based access), and a custom canvas UI — three non-trivial areas at once. That's why it's rated 7/10 rather than 6/10 like a typical 2-week project.

- **If Week 1 backend work is still incomplete by Day 8**, push the deadline to Day 17–18 rather than rushing the frontend — a half-working signature flow is worse than a complete but slightly late one.
- **Skip building your own file storage security from scratch** if it's eating too much time — a simple 'token = document access' check is enough for this project; you don't need enterprise-grade encryption.
- **Don't build email sending** unless you have time left over — copy-pasting the generated link manually is a perfectly fine substitute for a skill-testing project.

Milestone Checklist

Checkpoint	Milestone
End of Day 1	Server + DB connected, schema designed
End of Day 3	Auth + document upload working
End of Day 6	Token-based sharing + signature endpoint complete
End of Day 7	Backend fully tested — Week 1 complete
End of Day 9	Auth UI + upload/dashboard complete
End of Day 11	Recipients can open documents via secure link
End of Day 12	Canvas signature capture fully working
End of Day 13	Audit trail visible to owner
End of Day 14	Deployed, polished, documented — project complete

Skills This Project Will Test

- Handling file uploads correctly (validation, storage, size/type limits)
- Designing access control that doesn't rely purely on being logged in (token-based access)
- Working with the Canvas API to capture and convert user-drawn input to an image
- Structuring a database for something that needs a real audit trail
- Building a clean, minimal-but-real security model without overengineering it

What You Should Know Before Starting

- Basic JWT auth flow: issuing, verifying, and storing tokens
- How Multer handles file uploads in Express
- Basic exposure to the HTML Canvas API (mousedown/mousemove/mouseup drawing)
- Mongoose schema references and .populate()
- Comfortable generating and validating random tokens (crypto module)

Difficulty: 7 / 10 | Target Time: 14 days (extend to 17–18 if needed) | Stack: MERN + Tailwind CSS